RL-Cleanse: Learned Sequential Correction of Noisy IoT Sensor Streams Under a Strict Online Constraint

1st Anubhav Kiroula
dept. of Computer Science and Engineering
Graphic Era Hill University
Dehradun, India
anubhavkiroula1@gmail.com
ORCID: 0009-0008-6327-5390

2nd Archit Mittal
dept. of Computer Science and Engineering
Graphic Era Hill University
Dehradun, India
work.architmittal@gmail.com
ORCID: 0009-0005-1806-1033

3rd Chandradeep Bhatt
dept. of Computer Science and Engineering
Graphic Era Hill University
Dehradun, India
bhattchandradeep@gmail.com
ORCID: 0000-0002-7128-5862

4th Anupam Singh
dept. of Computer Science and Engineering
Graphic Era Hill University
Dehradun, India
anupamsingh@gehu.ac.in
ORCID: 0000-0002-3242-5383

***Abstract*—**Sensor streams at the IoT edge arrive one reading at a time. A device that must emit a corrected value immediately cannot buffer the future, and this paper measures what that costs. We formulate single-channel stream correction as a Markov decision process over a nineteen-dimensional scale-relative observation with six discrete corrective actions, then train a Double Deep Q-Network on 9,357 hourly readings from the UCI Air Quality record. Faults from five families are injected into a held-out chronological test split, and every method is scored on identical windows through one metric implementation. The online constraint proves to be an information barrier, not an inconvenience. A grid search over the rule filter's rejection threshold is monotone across the grid, so the calibrated filter degenerates to pass-through; on test it is bit-identical to inaction on all 150 windows. The learned policy does better, cutting mean absolute error to 0.2002 against 0.2170 and 0.4811 for two rule configurations, and a paired Wilcoxon signed-rank test confirms its advantage over the conventional filter on every fault family (pooled $p = 1.6\times10^{-17}$). Against inaction the picture is more complicated, and we report it in full. Pooled, the mean paired difference is −0.0167 with a bootstrap 95% confidence interval of [−0.0277, −0.0062]. The median is +0.0008 and the signed-rank test returns $p = 0.49$. Both numbers are correct: the policy helps on average while the typical window is unchanged. It improves spike and mixed windows, cannot be separated from inaction on drift, and significantly damages the two families whose uncorrected error was already near zero. Action traces explain why. The policy abstains on 85–97% of samples and corrects partially on the rest, sidestepping the accept-or-reject classification that the causal constraint makes ill-posed. We trace the over-correction to a missing action cost in the reward.

***Index Terms*—**Reinforcement learning, deep Q-network, Internet of Things, sensor data cleaning, streaming filtering, causal inference constraint, edge computing, anomaly correction.

I. INTRODUCTION

Deployed IoT sensing nodes produce telemetry that is corrupted before anyone analyses it. Electrical transients, chemical and thermal degradation of the sensing element, lost wireless packets, network retransmission glitches: each leaves its own signature in the record. Downstream analytics inherit all of them. Cleaning is a precondition for the inference built on top of it, not an afterthought.

Most published cleaning pipelines assume a setting that edge deployment does not provide. They want the batch: the full series read into memory, statistics computed across it, each suspect sample judged against its past and its future alike. A gateway forwarding a corrected reading the moment it arrives has none of that. At time t it holds the current packet and whatever it chose to retain. Nothing more.

This is not simply less context. It changes what is identifiable at all. Take a single-sample spike and the first sample of a genuine step change in the process. At the instant each arrives they are the same observation, a large innovation against the recent baseline. Only at t+1 do they separate, when one reverts and the other persists. A causal filter has to commit before that evidence exists. No filter design escapes this, because the limit sits in the information and not in the method, and it caps what every online approach can do.

This paper asks a narrow and answerable question:

Does a learned reinforcement-learning correction policy outperform static rule-based filtering when correcting noisy sensor readings under a strictly online, sequential constraint, in which the agent observes one reading at a time and cannot access the full batch?

We answer it on 9,357 hourly readings from a chemical multisensor array deployed at road level in an Italian city, comparing a Double Deep Q-Network against two configurations of a robust streaming filter and against doing nothing at all, on identical held-out windows, under identical constraints, through a single metric implementation.

Contributions.

1) We state the online constraint as an identifiability limit and verify it empirically. A grid search over the rule filter's rejection threshold on the training split is monotone in the threshold: the objective falls from 0.3724 at z = 4 to 0.2617 at

z = 20, the widest gate searched. The calibrated filter consequently degenerates to pass-through, and we confirm on test that it is bit-identical to inaction on all 150 windows. On this channel, at these fault amplitudes, no hard-thresholding filter earns its false-positive cost.

2) We show that a learned policy sidesteps the barrier by declining to make the classification. The policy abstains on 85–97% of samples and applies bounded partial corrections elsewhere, with an intervention rate that varies by fault family from 3.3% to 14.6%.

3) We supply paired significance testing in place of a comparison of means, and report where it contradicts the headline. The advantage over a conventional Hampel filter is significant on every family; the advantage over inaction is significant on two families, absent on one, and significantly reversed on two.

4) We report three negative results — over-correction of near-clean streams, a validation-to-test generalisation gap, and a corruption amplitude that lies inside the channel's own innovation envelope — and identify the missing action cost in the reward as the specific, testable cause of the first.

II. RELATED WORK

A. Data cleaning and imputation

Data cleaning is a mature database problem. Ilyas and Chu survey the field systematically [1], and Chu et al. set out the emerging challenges when cleaning is coupled to downstream machine learning [2]. ActiveClean [3] makes that coupling explicit, cleaning records progressively in an order chosen to benefit a convex-loss model while preserving convergence guarantees, and HoloClean [4] treats repair as probabilistic inference over integrity constraints and statistical signals. These systems are powerful and they are all offline: they assume random access to the relation. Our setting forbids it.

B. Robust filtering and streaming anomaly detection

The classical response to impulsive corruption is a robust filter. The Hampel filter replaces a sample with the local median only when that sample is an outlier by a scaled median-absolute-deviation test, and Pearson et al. characterise the generalized class, including recursive and cascaded variants [5]. Chandola et al. provide the standard taxonomy of anomaly detection [6], and Cook et al. survey the problem specifically for IoT time series [7]. The Kalman filter [8] remains the reference recursive estimator where process and measurement noise are Gaussian and their covariances are known. Our fault families violate those premises in three distinct ways, which we discuss in Section VII. Concept-drift adaptation [9] addresses non-stationarity, but its target is the model, not the individual reading.

C. Reinforcement learning and edge inference

Deep Q-learning [10] made value-based control viable on high-dimensional observations, and Double DQN [11] corrects the maximisation bias that inflates action values when the same network both selects and evaluates the greedy action. Sutton and Barto give the standard treatment of the underlying theory [12]. Henderson et al. document how sensitive reported deep-RL results are to seed choice and evaluation protocol [13], which motivates the multi-seed, held-out, significance-tested protocol we adopt here. On the deployment side, TensorFlow Lite Micro [14] establishes that networks of the scale we train are executable on microcontroller-class hardware.

D. The gap

Offline cleaning systems assume access this setting denies. Robust filters operate causally but, as we measure in Section VI-A, cannot beat pass-through on this channel at these amplitudes. Reinforcement learning has been applied to cleaning-adjacent problems, but we are not aware of work that states the online constraint as an identifiability limit, measures it, and then tests a learned policy against a baseline granted the same tuning budget under that constraint. That is the gap this paper addresses.

III. PROBLEM FORMULATION

A. The online constraint

Let y_t denote the reading received at step t and y^*_t the unobserved reference value. A correction policy π must emit a cleaned value $\hat{y}_t$ as a function of the information set $I_t = \{ y_1, \ldots, y_t \} \cup \{ \hat{y}_1, \ldots, \hat{y}_{t-1} \}$, and of nothing else. No element of $\{ y_{t+1}, y_{t+2}, \ldots \}$ is available, and no dataset-wide statistic that a deployed device could not itself compute may enter the observation. We additionally bound retained state to a constant: the policy keeps $N = 5$ previously emitted values, so memory is $O(1)$ in the stream length.

B. An identifiability limit

Any filter that decides whether to replace y_t must distinguish a corrupted reading from an unusual but genuine one. Write the innovation as the deviation of the current reading from the rolling baseline, scaled by the rolling dispersion. A single-sample additive spike of magnitude δ and a genuine step change of magnitude δ produce the same innovation at step t. The two hypotheses differ only in their continuation: the spike reverts at t+1 and the step persists. A causal filter must act at t.

The consequence is that a hard rejection rule faces an unavoidable trade: every threshold low enough to catch spikes also rejects genuine excursions. Whether rejection is worthwhile is an empirical property of the channel and the fault amplitude, and Section VI-A measures it for this channel. A policy with access to graded, partial corrections is not bound by the same trade, because it need not commit to a binary decision at all.

C. Markov decision process

We model the task as $M = \langle S, A, P, R, \gamma \rangle$ with the state, action and reward definitions that follow. The reward uses the reference signal and is therefore a training-time signal only; it is unavailable at deployment and never enters the observation.

IV. METHODOLOGY

A. Dataset

We use the UCI Air Quality record [15]: a gas multisensor device deployed at road level in an Italian city from March 2004 to April 2005, 9,357 hourly instances. The target channel is CO(GT), the ground-truth carbon monoxide concentration.

The published CSV encodes hardware faults with the sentinel −200. On CO(GT) that is 1,683 readings, 18.0% of the record. We resolve them by linear interpolation to obtain a continuous reference, then flag every interpolated position and exclude it from every metric. The reason is simple. Scoring a denoiser at a timestep whose reference came from an interpolator measures agreement with the interpolator, not with the sensor. Windows are additionally drawn to be at least 75% real measurement, a criterion applied identically to all methods and conditioned on no method's output.

TABLE I
CHRONOLOGICAL SPLITS

Split	Hours	Purpose
Train	6,737	Policy learning; rule-filter calibration
Validation	748	Checkpoint selection only
Test	1,872	Held out; used once, for the final table

Splits are chronological, as shown in Table I. We use neither shuffling nor k-fold cross-validation, because both leak future trend into past training instances and would invalidate any claim about streaming behaviour. The validation split is carved from the end of the training region and never from the test region.

B. Corruption model

The reference channel, once the sentinel positions are excluded, is treated as clean, and faults are injected synthetically. This is a deliberate methodological choice and its consequences are discussed in Section VII. It is forced by the requirement to compute error: no field IoT record ships a per-sample ground truth, so without a controlled corruption model no method — ours or the baseline's — could be scored at all. We inject four fault families plus a mixture, with the parameters given here so that the protocol is reproducible:

Electrical spike: each sample is corrupted independently with probability 0.08 by an additive offset of uniform magnitude 2.5–4.0 times the window standard deviation, with random sign.

Calibration drift: a linear ramp of slope 0.05–0.15 standard deviations per step, random sign, applied over a contiguous 12-sample sub-window.

Transmission dropout: contiguous runs of 1–4 missing samples, entered with probability 0.05 per step. The first sample of a window is never dropped, since a stream with no history at all is a cold-start problem rather than a denoising problem.

Duplicate packet: the previous reading is repeated with probability 0.06, modelling a network retransmission glitch.

Mixed: all four applied together at reduced rates (0.05, drift, 0.04, 0.04).

C. State space

The observation at step t is a vector in R^{19} assembled from the current packet and the five retained values: a reading-present flag; a signed logarithmic magnitude; a logarithmic dispersion term; four scaled deviations of the reading from the rolling mean, the rolling median, the previous emission and the mean-median gap; the history fill level; a time-of-day phase; episode progress; the five-element buffer expressed as scaled deviations from the rolling mean; and four heuristic fault flags.

Every deviation feature is divided by the rolling standard deviation, so the encoding is scale-relative. The policy therefore requires no dataset-wide normaliser that a device could not compute, and a unit test asserts invariance of the encoding under rescaling of the channel. Rolling statistics are computed over the last five values the agent itself emitted, never over raw future input.

Two invariants are enforced in code, not left to convention. No future access: the observation is constructed from the current sample and already-emitted values, and there is no code path to sample t+1. No oracle leakage: the generator's ground-truth, fault-type, reference-imputation and corruption-flag annotations are stripped before the observation is returned, so the agent receives exactly an identifier, a timestep, a sensor name and a value.

D. Action space

Six discrete actions are available, with semantics owned by the environment so that each has one definition and is unit-tested independently of any agent. They are listed in Table II. With no history available at step 0, every corrective action degrades gracefully to the raw reading.

TABLE II
ACTION SPACE

Action	Emitted value
a_0 skip	y_t unchanged (a dropout stays missing)
a_1 remove_outlier	rolling median
a_2 fill_missing	rolling median
a_3 fix_type	$0.5\, y_t + 0.5\, \mu_t$ (partial correction)
a_4 remove_duplicate	trend extrapolation from the last two emissions
a_5 fix_category	y_t clamped into the historical [min, max]

Two of the six are graded rather than binary. Action a_3 moves the reading halfway to the rolling baseline, and a_5 clamps it to the envelope the recent history has established. Neither requires the policy to decide whether the sample is corrupt, which is the decision Section III-B shows to be ill-posed under the causal constraint.

E. Reward

Let $e_{raw} = |y_t - y^*_t|$ be the error of taking no action, where for a dropout this is the error of the carry-forward value, and let e_{clean} be the error of the emitted value. With $\Delta e = e_{raw} - e_{clean}$, the reward is

$$R_t = R_{base} + \min(0.45,\ 0.2\,\Delta e) \quad \text{if } \Delta e > 0;$$
$$R_t = -0.15 - \min(0.35,\ 0.2\,|\Delta e|) \quad \text{if } \Delta e \le 0;$$
$$R_t = +0.15 \quad \text{for skip when } e_{raw} < 0.2.$$

Three properties are deliberate. The reward is magnitude-aware, so removing a large error pays more than removing a small one. It is a function of outcome only and never of the

generator's fault label, so the agent cannot learn to read a signal that will not exist at deployment. And both magnitude terms are clipped, so a single extreme spike cannot dominate an episode's return and destabilise the Bellman targets. Imputation of a dropout is rewarded exactly for beating carry-forward, not for emitting anything at all.

We note here, and return to it in Section VII, that this reward contains no cost for acting. A policy that intervenes on a stream requiring no intervention is penalised only through the error it introduces, which on a near-clean family is small in absolute terms. This is the specific cause of the over-correction reported in Section VI-C.

F. Agent and training

We train a Double DQN [11] with a multilayer perceptron of shape $19 \rightarrow 128 \rightarrow 64 \rightarrow 6$, totalling 11,206 parameters; Adam at 3×10^{-4}, batch size 64, $\gamma = 0.99$, Huber loss, gradient-norm clipping at 10, target network refreshed every 5 epochs, replay capacity 10,000, and 800 epochs of 24-hour episodes cycling the five fault families.

Double Q-learning is not a default here but a response to an observed failure. Under plain DQN the maximisation over the target network allows the same noise that inflates an action's estimated value to also select it, and behaviourally this appeared as a policy that over-corrected clean streams. We also record one deviation from our pre-registered plan: an ε-decay of 0.95 per epoch, applied literally, reaches the 0.02 floor at epoch 76, so an 800-epoch run would explore during only its first tenth. We derive the decay rate from the run length so that the schedule spans 70% of training, preserving the intended endpoints of 1.0 and 0.02. The literal value remains available as an explicit override.

G. Baselines

We compare against three references. Do nothing passes the stream through unchanged, carrying the last value forward across a dropout; it is reported as a column, not as an afterthought, because a cleaning method that cannot beat it is not worth deploying. Rule(cal) is a robust streaming filter operating under the identical causal constraint, with its rejection threshold, buffer length and warm-up period selected by grid search over $11 \times 3 \times 2$ configurations on the training split only — the same tuning budget the agent receives. Rule(txt) is the same filter at the conventional Hampel cut-off $z = 4$, representing what a practitioner would reach for without calibrating on this channel. Reporting a hand-picked threshold without calibration is how a strawman baseline is built, and reporting only the calibrated one hides what an uncalibrated deployment would do; we therefore report both.

H. Metric

All methods are scored through one implementation. Mean absolute error is computed over every timestep whose reference is a real measurement, with interpolated-sentinel positions excluded. Every method must emit a value at every timestep; an un-imputed dropout is charged for, not filtered out of the average. This convention matters: under the alternative, a policy that emits nothing at a dropout scores a perfect zero, which is an artifact and not a result.

V. EXPERIMENTAL SETUP

We evaluate on 30 held-out test windows of 24 hours per fault family. Window seeds are identical across methods, so every method sees the same windows with the same injected faults, and the per-window errors are paired observations. The RL column aggregates five independently trained seeds; per-window, the RL estimate is the mean across those five checkpoints, since five checkpoints are five samples of one training procedure rather than five methods.

For significance we use the Wilcoxon signed-rank test [16] on paired per-window differences, two-sided, with zeros dropped and the count reported. A t-test will not do here. The differences are heavily zero-inflated, with 61% of duplicate windows unchanged, and right-skewed by a minority of large transients. Normality is not on offer. Because each comparison repeats across five fault families, we apply the Holm–Bonferroni correction [17]. Beside each test we report a percentile bootstrap 95% confidence interval on the mean paired difference, over 10,000 resamples. The signed-rank test concerns the median of the differences; the benchmark headline is a mean. On this data the two disagree, and Section VI-C shows that the disagreement is itself the result.

VI. RESULTS

A. No finite threshold beats pass-through

Table III reports the best training-split objective achieved at each rejection threshold in the calibration grid. The sequence is monotone: widening the gate always helps, out to the widest value searched.

TABLE III
RULE-FILTER THRESHOLD SWEEP (TRAINING SPLIT)

z threshold	4	6	8	10	12	16	20
Best MAE	0.3724	0.2958	0.2826	0.2740	0.2727	0.2686	0.2617

The calibration therefore selects a gate so wide that it almost never fires. This is not a missing experiment but the honest outcome of calibrating: every narrower gate is worse. We verified the consequence directly on the test split — across all 150 windows the calibrated filter's per-window MAE is bit-identical to pass-through, with every paired difference exactly zero. On this channel, at these fault amplitudes, hard thresholding does not earn its false-positive cost, which is the empirical content of the identifiability argument in Section III-B.

B. Benchmark results

Table IV reports the main comparison. Relative MAE is each method's error divided by that family's own do-nothing error, so 1.000 means no improvement.

TABLE IV
HELD-OUT TEST RESULTS, 30 WINDOWS PER FAMILY, 5 SEEDS

Fault family	Do nothing	Rule(cal)	Rule(txt)	RL	RL std	RL rel
Electrical spike	0.2317	0.2317	0.4795	0.1608	0.0037	0.694

Fault family	Do nothing	Rule(cal)	Rule(txt)	RL	RL std	RL rel
Calibration drift	0.2743	0.2743	0.5351	0.2600	0.0015	0.948
Transmission dropout	0.0627	0.0627	0.2861	0.0946	0.0018	1.507
Duplicate packet	0.0299	0.0299	0.3351	0.0401	0.0036	1.344
Mixed real-world	0.4863	0.4863	0.7697	0.4457	0.0126	0.916
Mean	**0.2170**	**0.2170**	**0.4811**	**0.2002**	**—**	**1.082**

Two aggregates disagree and both are reported. Mean absolute MAE favours the learned policy: 0.2002 against 0.2170 for the calibrated filter and 0.4811 for the conventional one. The equal-weight mean of per-family relative errors does not: 1.082, slightly worse than inaction. The disagreement has a simple cause. The five families differ in error scale by more than an order of magnitude, from 0.03 on duplicate to 0.49 on mixed, so an unweighted mean of ratios gives the two near-clean families the same weight as the two that carry almost all the absolute error. Which aggregate matters depends on the deployment, and we decline to choose one on the reader's behalf.

One result is unambiguous in either aggregate: neither rule configuration improves on doing nothing. Rule(cal) is identical to inaction by construction, and Rule(txt) is between 1.6 and 11.2 times worse on individual families. A naively tuned industrial filter is actively harmful on this signal.

C. Significance testing

Table V reports paired Wilcoxon signed-rank tests with Holm correction and bootstrap confidence intervals. A negative mean difference favours the learned policy.

TABLE V
PAIRED TESTS, RL AGAINST EACH REFERENCE

Comparison / family	Mean diff.	95% CI	Median	p (Holm)
vs Rule(txt), pooled	−0.2809	[−0.362, −0.209]	−0.1338	1.6×10^{-17}
vs Rule(cal) ≡ do nothing				
Electrical spike	−0.0709	[−0.098, −0.045]	−0.0606	7.3×10^{-5}
Calibration drift	−0.0142	[−0.032, +0.001]	−0.0008	0.31 (n.s.)
Transmission dropout	+0.0318	[+0.018, +0.048]	+0.0151	7.1×10^{-7}
Duplicate packet	+0.0103	[+0.005, +0.017]	+0.0038	1.7×10^{-6}
Mixed real-world	−0.0406	[−0.070, −0.014]	−0.0329	0.035
Pooled, all families	−0.0167	[−0.028, −0.006]	+0.0008	0.49 (n.s.)

The central claim of the paper survives decisively. Against a conventional Hampel filter the learned policy wins on every family individually and pooled, at $p = 1.6\times10^{-17}$.

Against inaction the picture is mixed, and it has to be stated that way. Spike and mixed improve significantly. Drift does not. Its mean difference of −0.0142 carries a confidence interval spanning zero and a Holm-corrected p of 0.31, so the 5.2% improvement visible in Table IV cannot be separated from noise, and we withdraw it as a claim. Dropout and duplicate come out significantly worse, at p around 10^{-7} and 10^{-6}. That is systematic damage, not variance.

The pooled row is the most informative of the three, because its own statistics disagree. The mean paired difference is −0.0167, with a bootstrap interval of [−0.0277, −0.0062] that excludes zero, so on average the policy reduces error. The median difference is +0.0008 and the signed-rank test returns p = 0.49, so the typical window is untouched. Both are true. The aggregate benefit comes from a minority of windows holding large transients, where the absolute saving is big, set against a majority of small losses. One practical consequence is worth stating plainly: a single window picked at random is close to a coin flip and evidences nothing.

D. Per-window outcome distribution

Fig. 1 shows the 30 per-window differences behind each family mean. The distributions, not the means, are the finding.

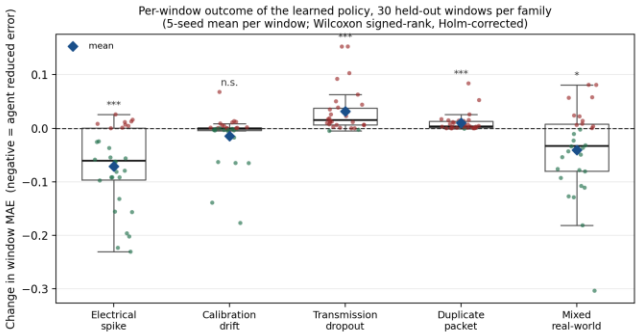


Fig. 1. Per-window change in MAE by fault family. Green points are windows the policy improved, red points windows it damaged, the diamond is the family mean. Asterisks mark statistical significance only; they do not indicate direction — dropout and duplicate are significantly worse.

Counting outcomes over all 750 episodes (30 windows × 5 seeds × 5 families), 34% of windows improve, 31% are unchanged and 35% are damaged. The split varies sharply by family: the policy improves 63% of spike windows and 65% of mixed windows, but damages 70% of dropout windows. Per-seed, pooled across families, only two of the five trained checkpoints reach significance against inaction on their own (p = 0.0023 and 0.0080); the other three do not (p = 0.23, 0.18, 0.13). We report this alongside the five-seed mean, which on its own would overstate how consistent the effect is.

E. What the policy actually learned

Table VI reports the action distribution over all 3,600 decision steps per family on the held-out windows.

TABLE VI
LEARNED ACTION DISTRIBUTION (% OF STEPS)

Family	skip	fix_type	fix_cat.	fill	other	Acts
Spike	92.5	1.8	2.4	1.7	1.5	7.5
Drift	93.5	2.2	1.9	1.0	1.3	6.5
Dropout	89.3	4.8	1.9	1.5	2.6	10.7
Duplicate	96.7	0.8	1.1	0.6	0.8	3.3
Mixed	85.4	6.1	3.4	2.0	3.0	14.6

The mechanism is learned selective abstention. The policy declines to act on 85–97% of samples and corrects the remainder. Its intervention rate tracks context: lowest on duplicate at 3.3%, where there is least to win, highest on mixed at 14.6%, which carries the largest absolute error. When it does act, it favours bounded, partial corrections. On the spike family it clamps to the historical envelope, which is the right shape of response to a value pushed outside the range recent history established; on dropout and mixed it moves the reading halfway to the baseline. Outright median replacement, the action closest

to what a Hampel filter does, is the least-used action on every family.

This is precisely the behaviour the causal constraint rewards. The policy is not solving the classification problem that Section III-B shows to be ill-posed; it is avoiding it, by acting rarely and by acting partially when it does. It also explains the failure mode: on the dropout family the policy acts on 10.7% of steps, close to the injected fault rate, but its imputations are worse than carry-forward, so a correct intervention rate paired with an inferior correction still yields net damage.

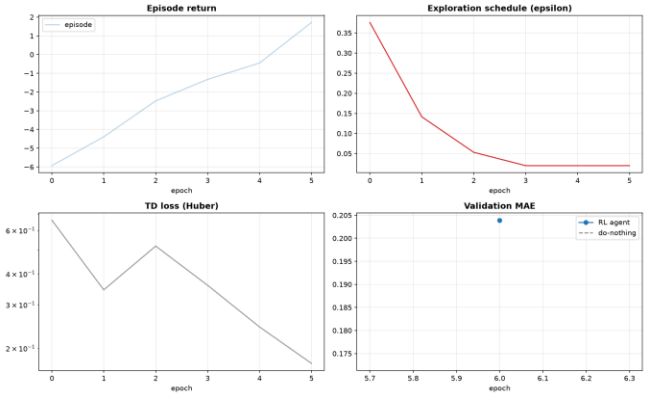


Fig. 2. Training curves across 800 epochs. Checkpoint selection uses the validation split only.

F. Validation did not predict test

Validation relative MAE averaged 0.9725 ± 0.0104 across the five seeds, spanning 0.9575 to 0.9858 — roughly a 3% improvement on inaction. Held-out test relative MAE averaged approximately 1.01, with per-seed values of 1.0172, 0.9977, 0.9994, 1.0245 and 1.0200. Selecting the best of about 32 validation evaluations per run is itself a selection bias, and this gap is a measurement of its size. The per-family test results are stable across seeds, with a spike-family standard deviation of 0.0037 on an MAE of 0.1608, but the aggregate claim of improvement over inaction does not survive the move to held-out data. We report the gap, not the validation figure alone.

G. Computational cost

Training runs 800 epochs in 83 seconds on an RTX 4050 Laptop GPU, and five seeds in approximately seven minutes. The resulting policy is an 11,206-parameter multilayer perceptron over a 19-dimensional input; inference is a single forward pass with O(1) memory, no lookahead and no dataset-wide normaliser, measured at 0.25 ms mean and 0.36 ms at the 95th percentile on the development host CPU. The pipeline auto-detects CUDA and falls back to CPU unchanged; unit tests and the reproduction notebook are CPU-only.

VII. DISCUSSION AND THREATS TO VALIDITY

A. The corruption model is ours

The faults are synthetic, and that is the most significant threat to validity in this paper. The reference channel is treated as clean and corruption is injected under the model of Section IV-B, so the results measure performance against that model rather than against field conditions. We adopt it for a blunt reason: computing error requires a per-sample reference, no field IoT record provides one, and the alternative is not a more realistic experiment but no quantitative experiment at all. Two qualifications are owed even so. The injected amplitudes are a parameter we chose, and behaviour at other amplitudes is unmeasured. The real channel is also not pristine once the sentinel positions are excluded: 18.0% of CO(GT) is a genuine hardware fault, and the clean channel's own innovations reach a 99th-percentile robust z-score of 8.8, with a maximum of 44 over 200 training windows.

B. The specified amplitude is marginal

That last measurement has a direct consequence. Injected spikes at 2.5–4.0 standard deviations land inside the channel's own innovation envelope, whose 99th percentile is 8.8. The faults are, by construction, hard to separate from genuine signal variation, and this caps what any causal method can achieve on this benchmark. We kept the amplitudes as specified instead of raising them to flatter the results, but a reader should not generalise the absolute numbers to a channel with a quieter baseline.

C. Over-correction and its cause

The policy significantly damages the dropout and duplicate families. Those percentages, −50.7% and −34.4%, look dramatic only because the denominators are near zero. In absolute terms the damage is +0.032 and +0.010, on a signal whose window standard deviation is around 1.1. The cause is identifiable rather than mysterious. The reward of Section IV-E carries no cost for acting, so a policy facing a stream that needs nothing has no positive incentive to stay still, and the skip bonus applies only below an absolute error of 0.2 in signal units. On a family whose entire uncorrected error is 0.03, the agent has nothing to gain and the reward never says so. Section IX specifies the remedy, which we regard as the clearest single path to making the aggregate claim hold.

D. Scope

This is a single dataset, a single channel, a single algorithm and five seeds. The state encoder is scale-relative by construction and a unit test asserts the invariance, but cross-channel transfer is untested. We have not compared against a Kalman filter. Its premises are violated here in three distinct ways: burst spikes are non-Gaussian, dropouts are not a noise process at all, and drift is a non-stationary mean shift, so fitting the process and measurement covariances on this channel becomes its own research problem. That is an explanation, not a defence, and we regard the missing baseline as a genuine gap. We have likewise not compared against a recurrent policy, so we claim the chosen architecture is suitable for the edge budget, not that it is superior.

E. Research-integrity note

An earlier revision of this pipeline produced a results table that no valid experiment supported. An audit identified five defects: exploration was permanently zeroed at epoch 10 by an

evaluation call, so the trained agent selected an unimplemented action on 110 of 120 steps and the reported RL output was in fact the raw stream; un-imputed dropout timesteps were filtered out of the error average, so a policy emitting nothing scored perfectly; the baseline fed its own replacements back into its history buffer and scored 0.208 MAE on perfectly clean input; and checkpoints were selected on the test split. A sixth defect was introduced during the repair and found afterwards: an unguarded checkpoint write caused the test suite to overwrite the published model with a 12-epoch artifact. All six are fixed and each has a named regression test asserting the specific failure. Both invalid-result episodes were caught by two independently computed figures disagreeing, not by code inspection, which we record because it is the only detection mechanism that worked.

VIII. PROPOSED EDGE DEPLOYMENT

The intended deployment target is a microcontroller-class sensing node. We state plainly that no physical hardware has been tested, and Table VII gives the per-component status so that the distinction between what is implemented, designed and planned is not left to the reader to infer.

TABLE VII
DEPLOYMENT COMPONENT STATUS

Component	Status
Policy training and evaluation	Implemented, reproducible
Serial / replay inference bridge	Implemented; replay-tested only
Circuit design and pinout	Designed, never wired
ESP32 firmware sketch	Written, not flashed
Simulation run	Planned; none recorded
On-device latency and RAM	Not measured

The feasibility argument is an argument, not a measurement. The policy is an 11,206-parameter network over a 19-dimensional input built from one packet and five retained floats. Inference is a single forward pass with constant memory and no lookahead, and the scale-relative encoder removes any need for a dataset-wide normaliser the device could not compute. Prior work establishes that networks of this size run on microcontroller-class hardware [14]. What we have actually measured is 0.25 ms mean inference on a development host CPU. That is a host number. An ESP32-class figure requires flashing the board and timing it there. One further gap is untested: the policy was trained on the UCI air-quality channel, not on the output of the metal-oxide sensor intended for the node, whose scale and noise characteristics differ. The scale-relative encoding is a reason to expect some transfer. It is not evidence of transfer.

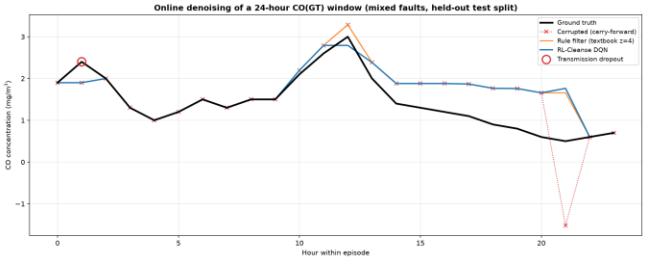


Fig. 3. A representative spike-family window: corrupted input, policy output and reference. The window shown is the one whose error reduction sits nearest the family median, not the best available window.

IX. CONCLUSION AND FUTURE WORK

We asked whether a learned policy beats static rule-based filtering for sensor-stream correction under a strictly causal constraint. Against rule-based filtering the answer is yes, and significantly so. The learned policy reduces mean absolute error to 0.2002 against 0.4811 for a conventional Hampel filter, with a paired advantage on every fault family at $p = 1.6\times10^{-17}$. The more interesting result is why the rule filter cannot win. We traced it to an identifiability limit of causal filtering, verified it with a threshold sweep that is monotone across the grid, and confirmed that the calibrated filter is bit-identical to inaction on every test window.

Against inaction the answer is conditional, and we have stated the condition. The policy significantly improves the two families carrying large transients, cannot be separated from inaction on drift, and significantly damages the two whose uncorrected error was already near zero. Pooled, the mean improvement is real while the median is not, because the benefit concentrates in a minority of windows. One mechanism produces both the gain and the failure. The policy learned to abstain on 85–97% of samples and to correct partially on the rest. That is the right response to a decision the constraint makes ill-posed. What it lacks is any incentive to abstain further on a stream that needs nothing at all.

Future work follows directly, in priority order. First, introduce an explicit action cost into the reward, or a learned abstention, and select its magnitude on the validation split rather than on test; this targets the over-correction directly and is the clearest single path to making the aggregate claim hold. Second, add Kalman-filter and offline-oracle baselines to bracket the learned policy from the classical and the acausal sides — an oracle-greedy policy permitted to see the reward already reaches 98.5% error reduction on spikes in our instrumentation, which quantifies what the online constraint costs and confirms that the action space is capable. Third, sweep the spike amplitude to map the detectability boundary and convert a stated limitation into a measured curve. Fourth, evaluate on a second channel with a lower sentinel rate to test the scale-relative encoder's transfer claim. Fifth, replace the feasibility argument of Section VIII with on-device latency and memory measurements.

REFERENCES

[1] I. F. Ilyas and X. Chu, Data Cleaning. New York, NY, USA: ACM Books, 2019.

[2] X. Chu, I. F. Ilyas, S. Krishnan, and J. Wang, "Data cleaning: Overview and emerging challenges," in Proc. ACM SIGMOD Int. Conf. Management of Data, 2016, pp. 2201–2206.

[3] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," Proc. VLDB Endowment, vol. 9, no. 12, pp. 948–959, 2016.

[4] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," Proc. VLDB Endowment, vol. 10, no. 11, pp. 1190–1201, 2017.

[5] R. K. Pearson, Y. Neuvo, J. Astola, and M. Gabbouj, "Generalized Hampel filters," EURASIP J. Advances in Signal Processing, vol. 2016, art. 87, 2016.

[6] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” ACM Computing Surveys, vol. 41, no. 3, pp. 1–58, 2009.

[7] A. A. Cook, G. Mısırlı, and Z. Fan, “Anomaly detection for IoT time-series data: A survey,” IEEE Internet of Things Journal, vol. 7, no. 7, 2020, doi: 10.1109/JIOT.2019.2958185.

[8] R. E. Kalman, “A new approach to linear filtering and prediction problems,” J. Basic Engineering, vol. 82, no. 1, pp. 35–45, 1960.

[9] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” ACM Computing Surveys, vol. 46, no. 4, pp. 1–37, 2014.

[10] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, no. 7540, pp. 529–533, 2015.

[11] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in Proc. AAAI Conf. Artificial Intelligence, 2016, pp. 2094–2100.

[12] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[13] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in Proc. AAAI Conf. Artificial Intelligence, 2018, pp. 3207–3214.

[14] R. David et al., “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in Proc. Machine Learning and Systems (MLSys), vol. 3, 2021, pp. 800–811.

[15] S. De Vito, E. Massera, M. Piga, L. Martinotto, and G. Di Francia, “On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario,” Sensors and Actuators B: Chemical, vol. 129, no. 2, pp. 750–757, 2008.

[16] F. Wilcoxon, “Individual comparisons by ranking methods,” Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.

[17] S. Holm, “A simple sequentially rejective multiple test procedure,” Scandinavian J. Statistics, vol. 6, no. 2, pp. 65–70, 1979.